

《人工智能通识十讲》

第七讲: AI Agent ——从工具调用到智能体时代

周天航

重质油全国重点实验室
中国石油大学 (北京) 未来能源学院

2026 年 9 月



先回顾: MCP 的“一个插座、三个角色、三类能力”

插座	MCP	开放协议, 统一模型与工具 / 数据的接法, 换模型不用重新布线
角色	Host / Client / Server	用户面对 Host, Host 为每个 Server 配一个 Client
能力	Tools / Resources / Prompts	可调用的函数 / 只读数据 / 预置模板

上一讲结束时, 工具柜已经装满: 查 DCS、查文献、算物性、发邮件
……一排排整齐地插在插座上。

可是还差一个人: **谁来决定何时用哪个工具、按什么顺序、什么时候停?**

今天的答案: 让模型自己来——这就是 **AI Agent (智能体)**。

情境：你的 AI 助手收到一封邮件——“Hi, 明天有空聚一下么?”

“机械化”的系统

生硬地回一句：“明天可以，请问几点?”

不知道你明天已排满，也不知道对方是谁。

像“人”一样的 Agent

检索日历 → 明天已满；翻历史邮件 → 定语气；识别出重要伙伴 → 要尽快敲定；调用 `send_email`：“嗨 Jim! 我明天排满了，周四上午有空，邀请已发。”

周四你突然发烧 → 自动改约：“非常抱歉，会面需推迟两天。”

课堂互动① · 举手投票 (1 分钟)

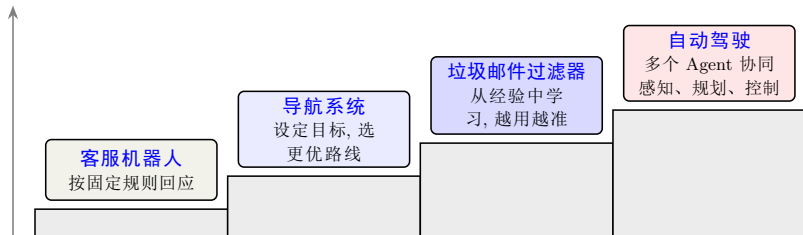
上一讲留的课前思考：你愿意让 AI “自己决定” 做几步？

A. 一步，每步都问我 B. 三步以内 C. 直到做完为止 D. 看是什么事：查资料随便，发邮件 / 花钱要问我

选 D 的同学说一句：你的分界线画在哪里？本讲最后一部分会回到这个问题。

“Agent”这个词，覆盖的范围很宽

自主程度



同一个词 “Agent”, 从 “按规则反应的小工具” 到 “会学习、会协作的智能体” ——全看你的需求

今天 DeepSeek、通义千问、ChatGPT 里的 **Deep Research** 已经会主动延伸查询、替你多翻几层资料，但它仍停留在**信息层面**，还没有替你**自主执行行动** ——还不算完整意义上的 Agent。

Q: 邮件助手、Deep Research、导航系统、自动驾驶——它们的“自主程度”差在哪一环？

本讲学习目标

知识目标

说清 Agent 与“聊天机器人”的根本区别; 记住六个模块和“感知—规划—行动—反思”循环; 知道多智能体为什么需要“总管”。

能力目标

能把一个多步任务拆成 Agent 的子任务并指定每步的工具; 能列出三种失败模式及对应护栏, 标出必须人工确认的步骤。

素养目标

理解“放权”与“负责”是一对: 让 AI 自己做事, 核对结果和承担后果的仍然是人。

学习建议: 全程带着两个化工例子——“实验方案设计 Agent”和“装置异常排查 Agent”, 每讲一个概念就问: 落在例子里是哪一步?

第一部分 · 什么是 AI Agent

AI Agent 是一种能够与环境交互、感知和收集信息，并利用这些信息执行自我驱动任务的软件程序

任务目标由人预先设定，但 Agent 能自主规划和选择行动路径以达成目标。

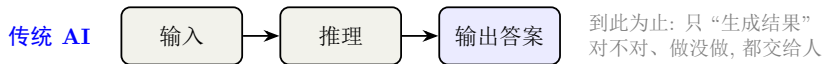
感知 它能“看到”环境：邮件、日历、屏幕、DCS 数据……

自主 路径不是人写死的，是它根据情况自己选的

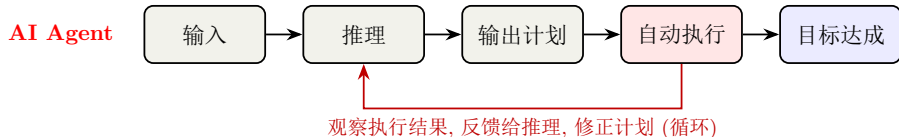
目标 但“要什么”始终由人定——这条边界本讲会反复回来

Agent 一词原意就是“代理人”：替你办事的人，不是替你拿主意的人。

传统 AI 与 AI Agent 的根本区别：能不能把“结果”变成“行动”



根本区别：能不能把“结果”变成“行动”，并在执行中根据反馈持续调整

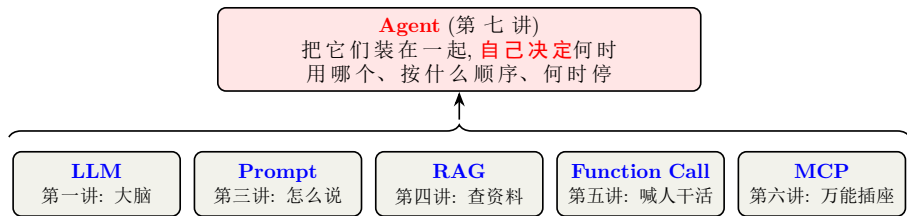


传统 AI 输入 → 推理 → 输出答案，到此为止

AI Agent 输入 → 推理 → 输出计划 → 自动执行，并在执行中根据反馈持续调整

Q:你让 DeepSeek 写了一段 Python 代码，然后自己复制去运行——这时候谁是“执行者”？

Agent 不是新发明: 前六讲的每一块, 都是它的零件



第二讲的 Token / 上下文长度贯穿始终: Agent 的每一步都在消耗上下文

LLM 是大脑, Prompt 是给它的目标, RAG 让它查得到资料, Function Call 让它喊得动人, MCP 让工具接得上。

把这些整合起来, 才构成一个**会理解、会规划、会调用工具完成任务**的智能体。

OpenAI 称 2025 年为“AI Agent 元年”; Google CEO 也说“我们已进入 Agentic Era”。口号听听就好, 关键看它到底能替你做什么。

回到邮件例子：四步里各用了哪个零件？

Agent 做了什么	用到的能力	哪一讲讲过
检索日历, 发现明天已满	外部知识 (RAG / 数据库)	第四讲
翻历史邮件, 判断语气	记忆 (跨会话的历史)	本讲第二部分
识别出重要伙伴, 要尽快敲定	推理与规划 (LLM)	第一、三讲
调用 <code>send_email</code> 回复	工具调用 (Function Call / MCP)	第五、六讲
周四生病 → 自动改约	反馈闭环 (新上下文 → 重新规划)	本讲第三部分

每一步单看都不新鲜, 新鲜的是: **没有人告诉它先查日历再翻邮件**, 顺序是它自己定的。

例子出自 Philipp Schmid 的 *Context Engineering* 一文, 第八讲还会再见到它。

从“轻量级”到“重型”：今天的 Agent 长什么样



轻量级 邮件助手: 靠上下文 + 几个工具调用, 只在自己的应用里动

信息型 Deep Research: 多轮检索、交叉比对, 产出报告, 但不替你“动手”

重型

OpenAI Operator、Anthropic Computer Use: 看屏幕截图, 输出鼠标 / 键盘动作, 在浏览器或操作系统层面直接干活

重型 Agent 被官方文档描述为“会看屏幕、能理解、会操作、能反复尝试直到完成任务的 AI 秘书”——核心是**截图—推理—动作—反馈**的闭环。

数字世界的“机械臂”： 鼠标 + 键盘

国内产品里,豆包、Kimi、通义的“深度研究 / 智能体”模式已经覆盖前两档。

化工里的 Agent 长什么样: 本讲的两个贯穿例子



第二讲问过: “三号反应器今晚温升为什么偏快?”

第二讲的“值班助手”只是回答问题; 今天的 Agent 会自己去查、去算、去写, 但最后一步都留给人。

例一: 实验方案设计 Agent

目标: “设计一个 Ni 基加氢催化剂的制备方案。”

它自己去: 查文献 → 提配方 → 算物性 / 热力学 → 评估安全 → 写实验步骤 → 交给导师确认。

例二: 装置异常排查 Agent

触发: DCS 报警 “R-101 床层温升偏快”。

它自己去: 读趋势 → 提出 3 个假设 → 逐个取证 → 排除 / 确认 → 给出建议和证据链 → 值班工程师决定是否动阀门。

判断题 (先举手, 再揭晓)

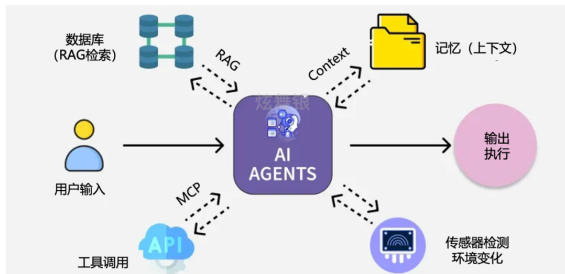
课堂互动②

下面四个, 哪些算今天说的 “AI Agent”?

1. 按关键词自动回复常见问题的客服机器人。
 2. 你让 DeepSeek 写了一段代码, 自己复制到电脑上运行。
 3. 值班助手每 5 分钟读一次 DCS 数据, 发现异常就自动调工具查趋势、写报告并通知你。
 4. 精馏塔回流量的 PID 控制器: 测温度、算偏差、调阀门, 一秒好几次。
-
1. ☒ 算, 但在阶梯最底层: 按规则反应, 自主程度很低。
 2. ☒ 不算, 执行者是你; 模型只输出了 “结果”, 没有把它变成 “行动”。
 3. ☒ 算: 有感知 (读数据)、有规划 (决定查什么)、有行动 (调工具)、有反馈 (异常消失才停)。
 4. 有感知、行动、反馈, 却没有 LLM 做规划——是经典控制意义上的 agent; 今天的 AI Agent 强调由大模型决定 “下一步做什么”。

判断标准: 是不是模型自己决定路径, 并把结果变成了行动。

第二部分 · Agent 的六个模块



配图: 用户输入进来, 经 RAG、记忆、工具、传感器四路支撑, 最后是“输出 + 执行”。

- | | |
|-----------------------------------|-------------------------------------|
| ① 用户输入 Prompt / Goal | ② 外部知识 Database / RAG |
| ③ 记忆 短期 / 长期 | ④ 推理与规划 Reasoning & Planning |
| ⑤ 工具调用 Function Call / MCP | ⑥ 反馈闭环 Feedback Loop |

模块①用户输入 · 模块②外部知识

①用户输入 (User Input) —— Prompt / Goal

所有任务的起点。用户用自然语言提一个请求, Agent 把它当作**目标**, 触发整个 workflow。

邮件例子: “帮我约一下 Jim 明天的会议”

化工例子: “设计一个 Ni 基加氢催化剂的制备方案”

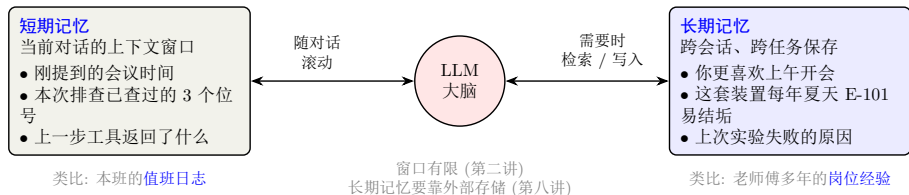
②外部知识 (External Knowledge) —— Database / RAG

Agent 不是“全知”的。日历、联系人、企业文档要靠**检索**拿到——第四讲的 RAG 在这里派上用场。

化工例子: 文献库、专利库、物性数据库 (沸点、闪点、生成焓)、实验室 SOP 文档、MSDS 安全数据表。

Q:第三讲说 Prompt 要写清“角色、指令、输入、输出格式”。给 Agent 的目标还要多写什么? (想想约束: 预算、截止时间、不许碰什么)

模块③记忆 (Memory): 短期记住“刚才”, 长期记住“一直”



除了查外部知识, Agent 还得“记住和你之前聊过什么”: **短期记忆**维护当前对话的上下文, **长期记忆**保存你的习惯与偏好。

第二讲说过上下文窗口是有限的——多数 Agent 今天只有短期记忆, 长期记忆怎么做, 是第八讲的主题。

模块④推理与规划 (Reasoning & Planning): Agent 的“大脑”

它决定**下一步做什么**。按复杂程度分三档:

档次	做法	化工例子
基于规则的反应	遇到 A 就做 B, 不思考	温度超限 → 发一条提醒
假设与推演	列出多种可能路径, 选最佳	温升偏快 → 进料变重? 氢量降? 阀故障?
拆解目标、排序	长任务链: 拆成子任务, 安排先后	制备方案: 先查文献, 再配方, 再算, 再审

大模型能做后两档, 靠的是第一讲讲到的“预测下一个词”加上第三讲的思维链:

先把思考写出来, 再决定动作——**写出来的思考就是计划**。

规划做得好不好, 是 Agent 之间拉开差距的主要地方; 第三部分专门讲它怎么转起来。

模块⑤工具调用 (Tools): 如果说 RAG 给了知识, 工具就是“手脚”

数字世界		化工实验室 / 装置	
send_email	发送邮件	search_literature	检索文献 / 专利
get_flight_info	查询航班	calc_properties	算物性 / 热力学 (反应热、平衡常数)
make_payment	调用支付	check_safety	查 MSDS, 判断闪点、相容性
鼠标 / 键盘 (Computer Use)	操作电脑	read_dcs_tag	读 DCS 位号趋势
电机 / 传感器指令 (机器人)	控制机体	write_report	按 SOP 模板生成实验步骤

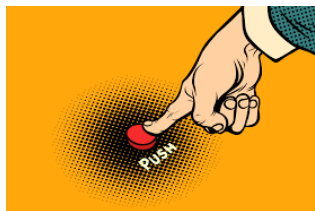
第五讲: 模型只输出“调用意图”, 执行在程序里; 第六讲: 用 MCP 把工具标准化, Agent 启动时[自动发现](#)有哪些工具。

注意表里没有 set_dcs_setpoint (改设定值)——哪些工具给 Agent、哪些不给, 是第五部分的话题。

模块⑥反馈闭环 (Feedback Loop): 行动 → 观察 → 再行动

Agent 不是“一次性”执行, 它在循环中不断调整:

- ▶ **物理世界**: 机器人尝试走路 → 观察姿态 → 调整动作;
- ▶ **数字世界**: Computer Use / Operator 点击网页 → 截图 → 判断下一步;
- ▶ **化工现场**: 排查 Agent 查完氢油比 → 发现降了 12% → 放弃另外两个假设, 集中取证。



按下去之后, 得看一眼发生了什么

执行结果 (如“航班已售罄”) 会**再次输入模型**, 模型据此重新规划: 换一个时间, 或者改高铁。

没有反馈的“多步执行”只是脚本; 有反馈才叫 Agent。

小组填表: 给“文献综述 Agent”配齐六个模块

课堂互动③

第四讲的实验室文献助手升级了: 老师只说一句“给我一份近三年渣油加氢催化剂失活机理的综述, 3000 字, 带参考文献”。

4 人一组, 3 分钟, 六个模块各写一句话; 写完交换检查, 挑出对方“最站不住脚”的一句。

- | | |
|--------|---|
| ①用户输入 | 目标是什么? 有哪些约束 (年份、字数、语言、只要 SCI)? |
| ②外部知识 | 从哪里检索? 2000 篇 PDF 的向量库够不够? 要不要联网? |
| ③记忆 | 一轮里要记住什么 (已读过哪些文献)? 跨任务要记住什么 (老师偏好的格式)? |
| ④推理与规划 | 先做什么后做什么? 读到一半发现主题太大, 怎么办? |
| ⑤工具调用 | 需要哪几个工具? 各自的输入输出是什么? |
| ⑥反馈闭环 | 怎么知道“写完了”? 参考文献是真的吗, 谁来核? |

六个模块, 对应前几讲的哪个词?

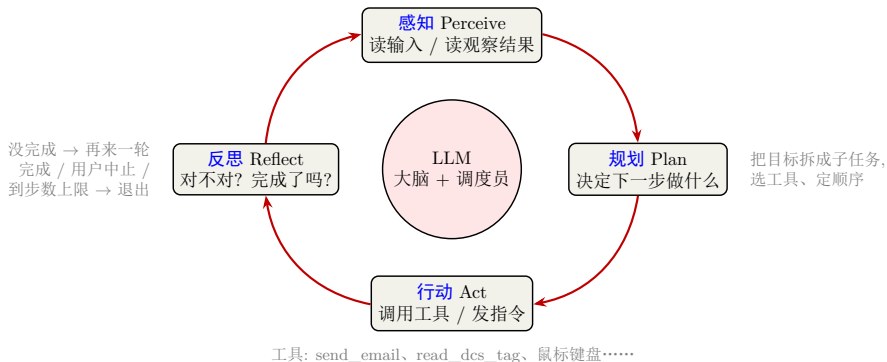
模块	对应的关键词	在 Agent 里的角色
①用户输入	Prompt (第三讲)	目标与约束
②外部知识	RAG (第四讲)	查得到
③记忆	上下文长度 (第二讲)、第八讲	记得住
④推理与规划	LLM + 思维链 (第一、三讲)	想得清
⑤工具调用	Function Call / MCP (第五、六讲)	做得到
⑥反馈闭环	本讲新内容	改得了

六个模块是“零件”。零件摆在桌上不会自己动——

Q: 它们是按什么节奏**转起来**的? ⇒ 第三部分: 感知—规划—行动—反思
循环

第三部分 · 感知—规划—行动—反思循环

环境: 邮件、日历、DCS 数据、屏幕截图……



LLM 在中间扮演“**大脑 + 调度员**”: 每转一圈, 读一次环境、做一次决定、动一次手、评一次结果。

上一讲“再循环一轮”说的就是它——只是这次, 循环几圈由模型自己定。

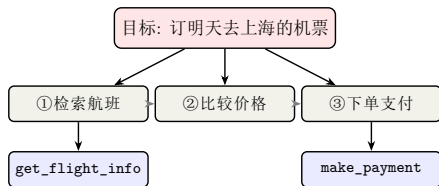
LLM 在一圈里做的四件事 (以“订明天去上海的机票”为例)

环节	做什么	机票例子
语义理解 → 目标提炼	把自然语言变成明确目标 + 约束	明天、上海、尽量便宜、上午到
任务分解 → 行动计划	拆成有序子任务	检索航班 → 比较价格 → 下单
工具调用 → 行为执行	每步决定调哪个 API / 函数	<code>get_flight_info</code> , <code>make_payment</code>
反馈循环 → 动态修正	结果再进模型, 重新规划	“航班已售罄” → 换时间, 或改高铁

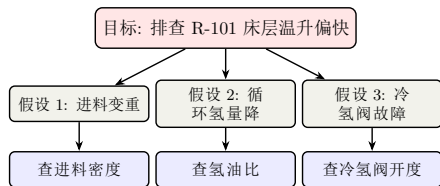
这四件事对应上一页的四个格子: 理解 $\approx$ 感知, 分解 $\approx$ 规划, 调用 $\approx$ 行动, 修正 $\approx$ 反思。

它不再只是“给出一个答案”, 而是像助理一样不断试错、修正, 直到完成目标。

规划的核心动作：把一个大目标拆成能执行的小步



数字世界: 拆成有序子任务, 每步选一个工具



化工现场: 先提出多个假设, 分别取证, 再选路径

左边是订机票的例子: 子任务有先后, 每步选一个工具。

右边是化工现场: 排查问题时往往**先并列几个假设、分别取证**, 再决定走哪条路——和你在实验室排故障的思路一样。

Q:如果三个假设都不成立呢? Agent 该做什么——再想新假设, 还是停下来找人?

ReAct: 让模型把“想”(Reasoning)和“做”(Acting)交替写出来

Thought: 温升偏快的常见原因有进料变重、循环氢量下降、冷氢阀故障。先查氢油比。

Action: `read_dcs_tag(tag = "FI-203", hours = 2)`

Observation: 循环氢流量 2 h 内由 800 降至 704, 下降 12%。

Thought: 假设 2 成立的可能性大。再查冷氢阀开度, 排除假设 3。

Action: `read_dcs_tag(tag = "HV-105", hours = 2)`

Observation: 开度稳定在 45%, 无异常。

Thought: 证据足够, 结束循环。

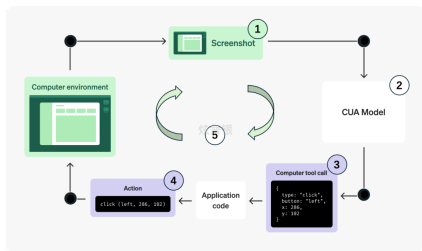
Final Answer: 建议提高循环氢量; 依据: FI-203 两小时趋势。请值班工程师确认。

每一段 Thought 是规划, Action 是行动, Observation 是感知, 下一段 Thought 里的“成立 / 排除”就是反思。

这就是 2022 年 [ReAct 论文 \(arXiv: 2210.03629\)](https://arxiv.org/abs/2210.03629) 提出的格式, 今天多数 Agent 框架仍在用它的变体。

位号、数值均为示意。

同一个循环, 换成“看屏幕、动鼠标”: Computer-Using Agent

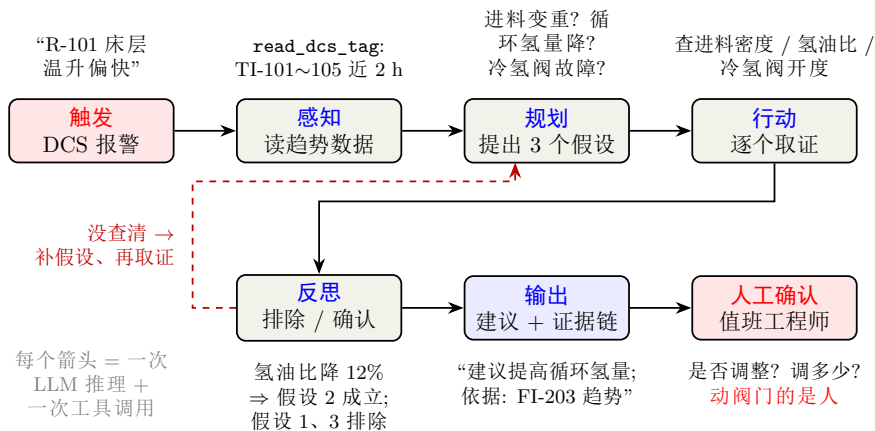


配图: 截图 → 模型 → 工具调用 → 动作 → 环境, 再截图

1. **输入:** 屏幕截图 + 光标位置 + 你的指令 (“帮我订一张飞上海的机票”);
2. **理解与推理:** 多模态模型认出按钮、表格、文字, 规划 “点哪里、输什么”;
3. **动作生成:** 输出一条指令, 如 `{"type": "click", "x": 286, "y": 102}`;
4. **执行:** 在虚拟机 / 浏览器沙盒里真的点下去;
5. **反馈:** 再截一张图, 判断完成没有; 没完成就下一轮。

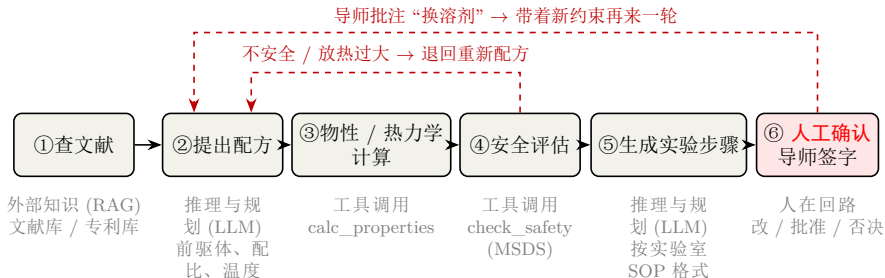
OpenAI Operator 与 Anthropic Computer Use 都是这个结构: 屏幕是它的“环境”, 鼠标键盘是它的“工具”。

化工例二：装置异常排查 Agent 的一轮多步推理



从报警到建议, Agent 跑了 3 ~ 4 圈, 每圈一次推理 + 一次工具调用; 输出的不只是结论, 还有可核对的证据链。

化工例一：实验方案设计 Agent 的六步流程



表面上是一条流水线，其实藏着两条回路：安全评估不过关 → 退回重新配方；导师批注 → 带着新约束再来一轮。

每一步各用一个模块：查文献是 RAG，配方是推理，算物性和查安全是工具，写步骤是推理，最后一步是人在回路。

Q: 导师改了一个字“换溶剂”，Agent 要从哪一步重跑？全部重跑，还是只跑③④⑤？

循环什么时候停？三个正常出口，一个兜底

出口	判断依据	化工例子
任务完成	反思环节判定目标已达成	证据链闭合，建议已生成
用户中止	人叫停，或人工确认点上被否决	导师：“这条路线别做了”
需要人介入	遇到不可逆动作、权限外的事、拿不准	要不要动阀门？停下来问
步数 / 预算上限	转了 N 圈还没完，强制退出	30 步还没查清 → 交给 人

每转一圈，上下文里就多一段 Thought / Action / Observation ——
第二讲的账单在这里**按圈数累加**：圈数越多，越贵、越慢、越容易忘掉开头。

“最后一个出口”不是失败，而是设计：第五部分会讲为什么它必须存在。

现场演示：数一数它转了几圈

课堂互动④

打开 **Kimi** 或 **DeepSeek** (开启联网 / 深度思考), 输入一个必须多步才能答的任务:

“查一下明天北京南到上海虹桥最早的高铁几点发车, 再按提前 40 分钟到站、从学校出发 1 小时车程, 告诉我几点出门。”

全班一起看屏幕, 2 分钟: 它做了几次搜索? 哪一步是感知、规划、行动、反思? 它有没有“反思”出自己第一次查错了?

看点:

- ▶ 它先拆任务了吗, 还是直接搜? (规划)
- ▶ 搜到的结果它有没有再核一遍? (反思)
- ▶ 最后一步“几点出门”是它算的, 还是猜的? 数字对吗? (核对)

联网搜索属于“信息型”自主; 它没有替你买票——想一想, 缺的是哪个模块? 还是缺的是权限?

第四部分 · 多智能体协作：一个人忙不过来，就组一个班组

一个 Agent 什么都干，会碰到三堵墙：

上下文墙 查文献、算物性、写报告的中间结果全塞在一个窗口里，很快就满了

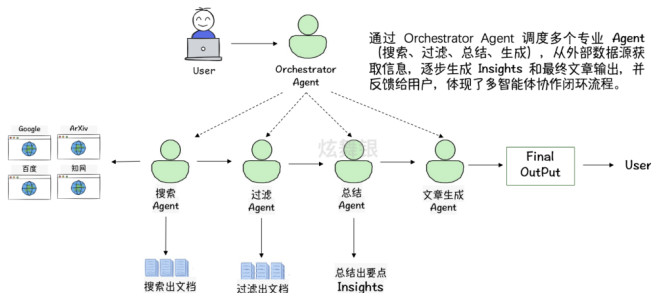
专业墙 一份 Prompt 既要懂催化剂又要懂安全规范，两头都不精

制衡墙 自己写、自己审，错了也看不出来

于是出现 **Agentic AI 系统**：多个 Agent 相互协作，通过交换信息实现协同——

每个 Agent 专注一个子任务，提升该领域的准确性与效率；
一个**编排 / 调度型 Agent (Orchestrator)** 统筹它们的调用顺序与数据流动。

一个例子: 一个 Orchestrator 带四个专业 Agent 写文章

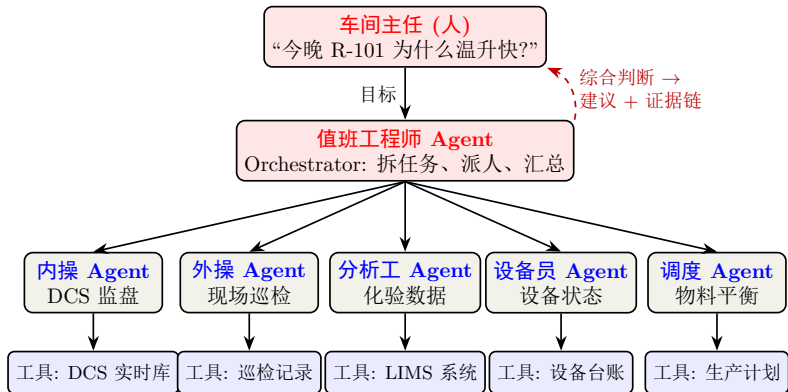


配图: 用户 → Orchestrator → 搜索 / 过滤 / 总结 / 文章生成四个 Agent → 汇总输出

搜索 Agent 连着 Google、arXiv 等数据源; 过滤 Agent 筛掉无关文档; 总结 Agent 提炼 Insights; 生成 Agent 落笔成文。

Orchestrator 自己不搜、不写, 只负责: 拆任务、派活、收结果、判断够不够、决定下一步找谁。

化工类比: 值班工程师 + 各岗位操作员

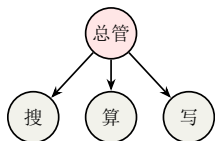


向下：分派子任务（“查一下氢油比”）

向上: 汇报带证据的结果 (“FI-203 两小时降了 12%”)

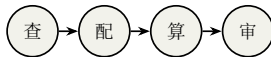
车间主任只对值班工程师说目标; 值班工程师不亲自监盘、巡检、化验, 而是派对的人去做对的事, 再把带证据的结论汇总上报。

多智能体的三种组织方式



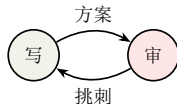
编排式

一个总管分派、汇总
例: 值班工程师 + 操作员



流水线式

上一个的输出是下一个的输入
例: 实验方案设计的六步



讨论式

一个写、一个审, 来回几轮
例: 方案 Agent vs 安全审查 Agent

方式	适合	要小心
编排式	子任务种类多、需要动态决定找谁	总管自己也会规划出错, 全队跟着错
流水线式	步骤固定、上一步产出就是下一步输入	前面一步错, 后面全错 (误差传递)
讨论式	需要制衡: 方案 vs 审查、写 vs 挑刺	来回轮数要封顶, 否则吵个没完

什么时候值得拆成多个 Agent? 别为了“多”而多

值得拆

- ▶ 子任务需要不同的工具或知识 (查文献 vs 算热力学);
- ▶ 中间结果很长, 一个上下文装不下;
- ▶ 需要互相制衡 (配方 Agent 与安全 Agent 不能是同一个);
- ▶ 子任务可以并行, 拆开更快。

不值得拆

- ▶ 三步以内就能完成的小任务;
- ▶ 子任务之间要频繁来回传大段上下文——通信开销比省下的还多;
- ▶ 只是想让系统“看起来更高级”。

多一个 Agent, 就多一份 Prompt 要维护、多一处会出错、多一倍 token 账单。
先用一个 Agent 跑通, 撞到墙了再拆 —— 和第二讲“先 Prompt、再 RAG、最后微调”是同一种工程直觉。

案例分析 + 角色扮演: 组一个“催化剂制备方案”Agent 班组

课堂互动⑤

4 人一组, 5 分钟。分别扮演 总管 Agent、文献 Agent、配方 / 计算 Agent、安全审查 Agent。目标: “设计一个 Ni 基加氢催化剂的浸渍法制备方案”。

请把对话演出来: 总管先派谁? 派活时给对方什么信息? 安全 Agent 什么情况下把方案打回? 打回后总管找谁重做? 最后谁把方案交给导师?

演完回答:

1. 你们用的是编排式、流水线式还是讨论式? 还是混合的?
2. 哪一次“传话”最容易丢信息? (例: 文献 Agent 说了“焙烧 450 度”, 配方 Agent 记成了 550)
3. 如果把安全 Agent 去掉, 让配方 Agent 自己审自己, 会少发现什么?

选 1 组上台演 2 分钟, 其他组当“导师”: 你敢签这份方案吗? 为什么?

第五部分·失败模式与护栏：放权之前，先想好它会怎么出错

Agent 拿到了这么多上下文和工具，会不会“记忆错乱”？——会，而且这只是问题之一。

三大问题	表现	化工场景里会怎样
幻觉与推理错误	长任务链里推理失误，工具调用失败或结果不稳定	方案里引用了不存在的文献，闪点算错一个数量级
短期记忆限制	只记得眼前对话，记不住长期偏好和跨任务经验	上次实验失败的原因，这次又踩一遍
缺乏适应能力	新场景下表现脆弱，人能随机应变，它不能	DCS 换了位号命名规则，它就全查空

工程上，这些问题最常以三种**失败模式**冒出来：**死循环**、**幻觉调用**、**成本失控**。逐个看。

失败模式一：死循环——同一个动作反复做，目标却不前进

```
Action: search_literature("Ni 催化剂失活 2024") → Observation:
0 条结果
Thought: 换个关键词。 Action: search_literature("nickel catalyst
deactivation 2024") → 0 条
Thought: 再 换。 Action: search_literature("Ni-based catalyst
deactivation") → 0 条
Thought: 再 换。 Action: search_literature("Ni catalyst
deactivation review") → 0 条
……(第 31 次) ……
```

真正的原因可能是**文献库服务掉线了**，而不是关键词不对——但模型看到的每一次 Observation 都“合理”，于是一直换词。

化工类比 操作员发现流量不涨，一直开大阀门，却没去看是不是泵停了

护栏 **步数上限** (最多 N 圈) + **重复检测** (同一工具、相近参数连续出现 3 次即中止，报告给人)

失败模式二：幻觉调用——编工具、编参数、编结果

编什么	例子	后果
编工具	调用了不存在的 <code>get_msds_pdf</code>	程序报错；差的框架会让模型“假装成功”继续
编参数	把 450 °C 当成 450 K 传给 <code>calc_properties</code>	反应热算错，后面的安全评估全部失真
编结果	工具超时没返回，模型自己“补”了一个焓值	实验步骤里出现一个谁也查不到来源的数字

第三讲讲过模型会“一本正经地胡说”；在 Agent 里，胡说会**顺着工具链往下传**，越传越像真的。

护栏 **工具白名单** (只能调注册过的工具) + **参数校验** (单位、范围、类型) + **报错必须处理** (不许跳过)

化工类比 操作票上的每个位号、每个数值，都要有人复核签字

失败模式三：成本失控——上下文越滚越长，花费按平方增长

每转一圈，之前所有的 Thought / Action / Observation 都要再读一遍。
算一笔账：

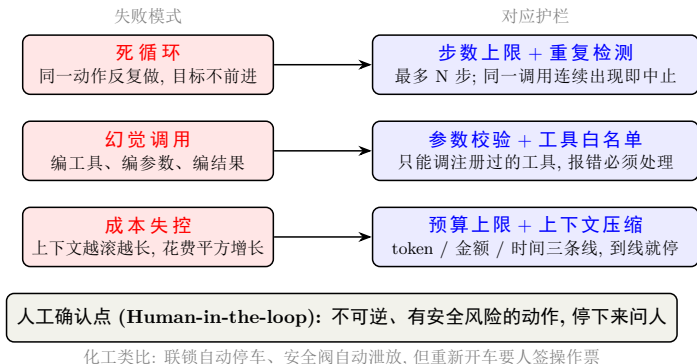
每圈新增上下文	约 2K token (一次工具返回 + 一段思考)
跑 10 圈的总输入	$2K \times (1 + 2 + \dots + 10) = 110K \text{ token}$
跑 30 圈的总输入	$2K \times (1 + 2 + \dots + 30) = 930K \text{ token}$: 圈数 $\times 3$, 费用 $\times 8.5$
按输入 2 元 / 1M tokens	一次 30 圈约 1.9 元 ; 值班助手一天跑 1000 次, 就是 1900 元

价格为示意, 可查 [DeepSeek API 文档](#) 定价页; 有缓存时更便宜, 但平方增长的趋势不变。

护栏 **预算上限** (token / 金额 / 时间三条线, 到线就停) + **上下文压缩** (把前几圈总结成一段, 第八讲详讲)

化工类比 公用工程也有配额: 蒸汽用超了, 调度会先打电话问你在干什么

三种失败模式, 三道护栏, 再加一条底线



前三道护栏是**自动的**, 写在 Agent 框架里; 最后一条是**人工的**: 不可逆、有安全风险的动作, 停下来问人。

化工人对这套设计不陌生——**自动保护兜底, 关键动作由人签字。**

人工确认点放在哪？给动作分三级

等级	判断标准	实验室 / 装置里的例子
自动放行	只读、可重做、不花大钱	查文献、读 DCS 位号、算物性、查 MSDS
需人确认	对外可见、花钱、会写入系统	发邮件给供应商、下单买试剂、把方案写进 ELN 电子记录本
不给工具	不可逆、有安全后果	改 DCS 设定值、启停设备、开始加热反应釜——只输出建议，由人操作

这正是第六讲“Resources 可自动放行、有副作用的 Tool 要确认”那条规则，搬到了 Agent 上。

回到开场的投票：选 D 的同学画的那条线，大致就是第一级和第二级之间；

而第三级的线，不该由 Agent 或用户临时决定，应该在设计时就写死。

素养与责任

负责任地放权：目标你定，责任也在你

Agent 替你做了十步，出了问题，署名的还是你。四条具体做法：

- ▶ Agent 生成的实验方案，**每一条参考文献都要点开核对**是否真实存在、是否真的这么说——编造的引用写进论文就是学术不端；
- ▶ 给它**预算和步数**，像给实验申请经费一样：花多少、跑几圈，事先说清；
- ▶ 不可逆的动作（加热、投料、改设定值）**永远留给人**，哪怕它的建议看起来很对；
- ▶ 报告里**写明哪些部分由 AI 完成、用了什么工具**，让读的人知道该核对什么。

Q:讨论 (2 分钟): 如果排查 Agent 建议“提高循环氢量”，值班工程师照做后出了事故，责任在谁？

提示：想想操作票制度为什么要求“建议人”和“执行人”都签字。

往前看: Agent 下一步要补的课, 是“记住该记住的”

人脑的本事: 很久以前的事, 需要时能精准提取。未来的 Agent 也需要类似的**长期记忆机制**, 逐渐形成自己的“用户画像”。



配图: ChatGPT 的 Memory 设置页, 保存了跨对话的用户偏好

- **模型层面**: 与稀疏注意力的思路相似——不是全塞进来, 而是学会记住该记住的;
- **架构层面**: 像 ChatGPT 的 Memory 模块, 把高相关内容长期保存, 后续任务里直接用。

化工版: 记住“这套装置每年夏天 E-101 易结垢”、“导师不喜欢用含氯溶剂”——下一讲 Context Engineering 讲怎么做。

小组讨论: 给“实验方案设计 Agent”写一张护栏卡片

课堂互动⑥

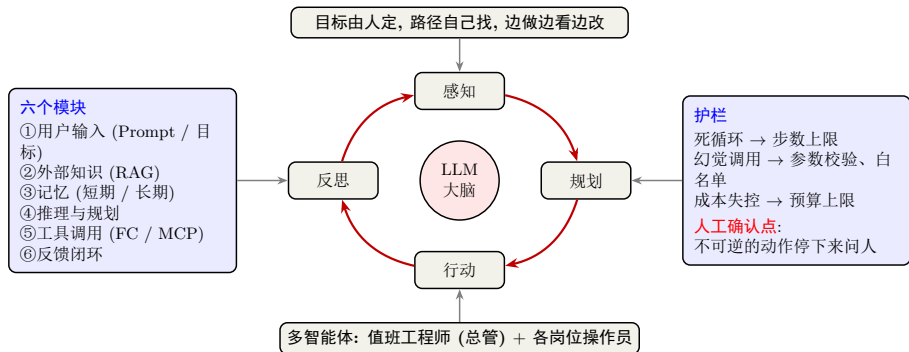
4 人一组, 4 分钟, 填一张卡片交上来:

- ▶ 步数上限: 最多转几圈? 为什么是这个数?
- ▶ 预算上限: 一次方案设计最多花多少 token / 多少钱 / 多长时间?
- ▶ 工具白名单: 哪 4 ~ 5 个工具可以调? 哪一个坚决不给?
- ▶ 人工确认点: 六步里哪几步必须停下来问人? 问谁 (学生 / 导师 / 安全员)?

交卡片后, 两组互换, 各挑一条: “这条护栏太松, 会出什么事?” 或 “这条太紧, 会把 Agent 卡死在哪一步?”

护栏不是限制能力, 是让你敢把更多的事交给它。

本讲总结：一个循环、六个模块、一组护栏



目标由人定, 路径自己找, 边做边看边改; 零件来自前六讲, 新东西是循环和护栏; 事情多了就组班组, 但总管只有一个。

七个词, 各用一句话带走

Agent	目标由人定, 自己规划路径、调用工具、把结果变成行动的程序
感知—规划—行动—反思	每转一圈: 读环境、定下一步、动一次手、评一次结果
ReAct	把“想”和“做”交替写出来: Thought / Action / Observation
记忆	短期在上下文窗口里, 长期要靠外部存储 (第八讲)
工具	Agent 的手脚; 给哪些、不给哪些, 是设计时定的
Orchestrator	多智能体的总管: 拆任务、派活、汇总, 自己不干活
护栏	步数 / 预算 / 白名单自动兜底, 不可逆动作留给人

Q:回到开场: 那封“明天有空聚一下么”的邮件, 现在你能说清 Agent 转了几圈、哪一步该问你了吗?

随堂快测

课堂互动 · 3 道题

- Q1 (单选) 下面哪一条最能区分 “AI Agent” 和 “聊天机器人”?
- A. 参数更多 B. 能把结果变成行动, 并根据反馈调整 C. 上下文更长 D. 接了 MCP
- Q2 (判断) 给 Agent 设步数上限会限制它的能力, 所以正式系统里不应该设。
- Q3 (简答) 排查 Agent 建议 “提高循环氢量”, 为什么最后动阀门的必须是人? 给两个理由。

答案: Q1 选 **B**; Q2 **×** 错误, 上限是防死循环和成本失控的护栏, 到线交给
人不是失败;

Q3 动阀门不可逆、有安全后果 (第三级动作); Agent 可能幻觉调用或推理
出错, 证据链要人核; 责任主体必须是人。

线上提交, 当堂出分。

课后任务与拓展

- 必做① 记轨迹: 在 [Kimi](#) 或 [DeepSeek](#) 里让 AI 完成一个需要至少 3 次搜索的任务 (自选主题), 截图记录它的每一步, 用“感知 / 规划 / 行动 / 反思”给每步贴标签, 并指出一处它本该反思却没有反思的地方 (300 字以内)。
- 必做② 写护栏: 为“装置异常排查 Agent”写一页护栏清单: 步数上限、预算上限、工具白名单 (标出只读 / 需确认 / 不给)、人工确认点各在哪一步, 每条给一句理由。
- 选做③ 动手搭: 在 [阿里云百炼控制台](#) (参考 [百炼文档](#) 智能体应用章节) 创建一个智能体, 挂上至少一个工具或知识库, 截图它一次多步调用的过程。

拓展阅读: [ReAct 论文 \(arXiv: 2210.03629\)](#) 的第 1、3 节 (选读); [MCP 官方仓库](#) 里看看 Agent 是怎么“自动发现”工具的。

下讲预告: Context Engineering —— AI 如何记住重要信息

Agent 最大的瓶颈是记忆

本讲的每一圈都在往上下文里塞东西, 塞满了怎么办

上下文不是越多越好

第二讲说过“稀释注意力”, 现在要学会主动取舍

该记什么、存哪里、何时取

短期记忆压缩、长期记忆写入、需要时检索

邮件例子的另一半

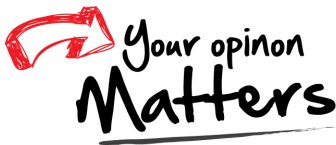
Agent 怎么“知道” Jim 是重要伙伴、你偏好上午开会

化工版用户画像

让值班助手记住“这套装置每年夏天 E-101 易结垢”

Q:课前思考: 和 AI 聊了一个学期, 你最希望它“记住”你的哪一件事? 又最不希望它记住哪一件?

Thank You!



第七讲 · AI Agent ——从工具调用到智能体时代 周天航 · 中国石油大学 (北京)

下课。请完成课后任务,下次课把你“最希望 AI 记住 / 最不希望它记住”的那件事带来。